

Iterative Sparse Matrix–Vector Computation on Evolving Sparse Matrices

One-Shot Project Report

Nicola Annunziata

October 2026

Contents

1	Project Overview	1
2	Implementations	1
2.1	C++ Threads Implementation	1
2.2	OpenMP Implementation; Task Based & Worksharing	1
2.3	MPI+OpenMP Implementation	2
3	Correctness	2
3.1	C++ Threads and OpenMP Correctness	3
3.2	MPI+OpenMP Correctness	3
4	Results	3
4.1	Shared-memory execution time and speedup	4
4.2	Granularity Analysis	4
4.3	Comparison between regular and irregular workloads	6
4.4	Strong Scaling	7
4.5	Weak Scaling	8
4.6	OpenMP task-based vs OpenMP work-sharing	10
4.7	MPI+OpenMP configuration sweep	11
5	Conclusions and limitations	12
5.1	Conclusions	12
5.2	Limitations and bottlenecks	12

1 Project Overview

The project investigates the parallelization of an iterative algorithm based on sparse matrix-vector multiplication (SpMV). The matrix $A \in \mathbb{R}^{N \times N}$ is stored in compressed sparse row (CSR) format and, starting from an initial vector $x_0 \in \mathbb{R}^N$, SpMV and normalization operations are performed iteratively:

$$y_k = A_k x_k, \quad x_{k+1} = \frac{y_k}{\|y_k\|_2}.$$

At regular intervals, the matrix structure evolves through a circular shift of the rows. The matrix dimensions and the total number of nonzero elements remain constant, but the mapping between output rows and source CSR rows changes.

The implementations compared are:

- a persistent pool of C++ threads with dynamic chunk acquisition;
- OpenMP with explicit tasks and, for comparison, dynamic work-sharing;
- a hybrid MPI+OpenMP program for multiple compute nodes.

Compact affinity was configured for OpenMP; for C++ Threads, CPU resources were allocated through Slurm. Physical cores were used for up to 16 threads, while logical CPUs were also used with 32 threads. Placement was configured using OpenMP affinity variables and Slurm binding options.

2 Implementations

2.1 C++ Threads Implementation

The C++ Threads implementation uses a fixed-size *ThreadPool*, created only once and reused throughout the entire iterative computation. When the task queue is empty, workers are suspended using a `condition_variable` and awakened when new work is added, thus avoiding busy waiting and unnecessary CPU consumption, while access to the shared queue is protected by a `mutex`, which ensures mutually exclusive access when tasks are inserted and retrieved. Each task is associated with a `future` through a `packaged_task`, allowing the main thread to wait for the parallel jobs to complete before proceeding to the next phase. SpMV uses dynamic work distribution. The matrix rows are divided into chunks whose size is configurable through the `-b` parameter, whose default value is defined in the common configuration file used for the experiments, and for each SpMV only as many jobs as there are workers are submitted to the pool. Each worker then acquires the next chunk by atomically incrementing the shared `next_chunk` counter, until all available work has been exhausted. This approach allows threads that finish earlier to continue processing new blocks, making it particularly useful when rows have an irregular number of nonzero elements.

Operations with uniform cost, such as the dot product and vector scaling, instead use a static partition into contiguous ranges. In the dot product, each worker computes an independent partial sum; the results are stored in structures aligned to 64 bytes to limit *false sharing* and are finally combined sequentially. Matrix evolution does not require copies or physical movement of rows: `row_shift` only changes the mapping between a logical row and the original CSR row during SpMV.

2.2 OpenMP Implementation; Task Based & Worksharing

Two variants were developed for OpenMP: one based on *tasks* and one based on *work-sharing* directives. Both use a single `omp parallel` region, keeping the same team of threads active throughout the entire iteration

In the *task-based* version, a thread within `omp single` controls the algorithm’s flow and generates the parallel work. Dot product and scaling preserve the static partition of the C++ Threads version, creating one task for each partition, while SpMV generates one task for each chunk of rows. The block size therefore directly determines the granularity and number of tasks. Task assignment to workers is delegated to the OpenMP runtime, eliminating the `next_chunk` atomic counter, synchronization between the various phases is ensured through `omp taskwait`, which replaces the explicit wait on the `future` objects used in the C++ Threads version, so that normalization cannot begin until all chunks have been completed

The *work-sharing* variant completely eliminates explicit task creation: all threads cooperate directly through `omp for` constructs. SpMV uses `schedule(dynamic, chunk_size)`, so the OpenMP runtime directly assigns blocks of rows to threads dynamically. This approach is suitable for the irregular matrix because a thread that finishes its block can receive another without waiting for the others. Dot product and scaling use `schedule(static)`. In the dot product, each thread accumulates a partial result in a structure aligned to 64 bytes; the `nowait` clause removes the loop’s implicit barrier, which is replaced by an explicit barrier before the final reduction performed in `omp single`. Finally, `single` regions are reserved for shared serial operations, such as updating the row shift, computing the norm, swapping the vectors, and computing the final checksum.

2.3 MPI+OpenMP Implementation

The distributed version combines MPI, used to divide the work among the different processes, and OpenMP, used to parallelize computation within each process. In this way, each process works on its own portion of the matrix using multiple threads in parallel.

The matrix is generated by rank 0 and divided into contiguous row ranges. The partition is obtained not by dividing the number of rows uniformly, but by using the cumulative number of nonzero elements in the CSR representation, so as to assign each rank an approximately balanced amount of work. Rank 0 therefore computes, for each process, the number of assigned rows and the starting position of the corresponding matrix portion, distributes them through `MPI_Bcast`, and uses `MPI_Scatterv` to send each process the local portions of `values`, `col_idx`, and `row_ptr`. The latter is then adjusted locally by subtracting the partition’s initial offset.

During each iteration, each rank performs SpMV only on its own rows, parallelizing it through OpenMP tasks. To compute the squared norm, an `MPI_Allreduce` was used at each iteration to combine the contributions computed locally by the different processes and obtain the global norm.

An `MPI_Allgatherv` was then used at each iteration to reconstruct the global vector required for the next computation, plus an additional final `MPI_Allgatherv` to compute the Rayleigh value. The row shift is handled logically, without redistributing the matrix among the processes at each epoch. After the global vector has been reconstructed, it is rotated according to the value of `row_shift`, producing the same effect as shifting the matrix rows.

To analyze performance in greater detail, the times for initial distribution, SpMV, normalization, rotation, communication, reduction, and epoch transition were also measured separately. For each phase, rank 0 considers the maximum time recorded across all processes.

3 Correctness

Verification was conducted using two complementary approaches: *strong verification* and *weak verification*, used to check the consistency of the results obtained by the different parallel versions against the reference implementation.

Tolerances are necessary because parallel execution can change the order of floating-point operations, producing very small numerical differences while preserving the correctness of the

result. The checksum was used as an additional diagnostic check: it can differ even when the vectors are numerically equivalent within the specified tolerance.

3.1 C++ Threads and OpenMP Correctness

The correctness of the C++ Threads and OpenMP implementations was verified by comparing their results with the sequential version. In both cases, the Rayleigh value, the checksum, and the final vector obtained through `-dump-vector` were checked, with the vector compared element by element within a numerical tolerance.

For C++ Threads, the test was performed with $N = 2000$, $NZ = 20000$, *irregular* mode, and block size 16. With 1 thread, the result matched the sequential result, while with 4 threads, the difference in the Rayleigh value was approximately $5 \cdot 10^{-15}$ and the maximum error in the vector was approximately $9.7 \cdot 10^{-17}$. The comparison was performed with an absolute tolerance of 10^{-12} and a relative tolerance of 10^{-10} .

The OpenMP *task-based* and *work-sharing* versions were tested with $N = 5000$, $NZ = 20000$, *irregular* mode, 4 threads, and block size 256. The Rayleigh value differs from the sequential value by approximately $8 \cdot 10^{-15}$; the vector comparison, with a tolerance of 10^{-12} , yielded a maximum absolute error of approximately $9.02 \cdot 10^{-17}$ and a maximum relative error of approximately $1.98 \cdot 10^{-15}$, obtaining **PASS** for both versions.

3.2 MPI+OpenMP Correctness

The correctness of the MPI+OpenMP version was verified by comparing the final Rayleigh value with that obtained by the sequential version, keeping the problem size and mode unchanged.

For the small problem, with $N = 1000$, $NZ = 1000$, and *irregular* mode, the sequential version produced 0.752534652397283. Different MPI+OpenMP configurations returned values between 0.752534652397281 and 0.752534652397282, with a maximum difference of approximately $2 \cdot 10^{-15}$ from the reference.

Verification was also repeated on the main problem, with a matrix of size $N = 10^6$, $NZ = 2 \cdot 10^8$. The sequential Rayleigh value was 9.550185371928109, while execution with 8 ranks and 16 threads produced 9.550185371928444, with a difference of approximately $3.35 \cdot 10^{-13}$, obviously the configuration was run using one rank per node, utilizing the 16 physical cores available on each node.

For the small problem, the final vectors were also generated through `-dump-vector`, while the checksum was retained as an additional diagnostic check.

4 Results

Each configuration was run three times, and the median time was used to reduce the influence of variability across individual runs. Matrix generation time was measured separately and is not included in computation time.

Unless otherwise specified, the reference configuration uses $N = 10^6$, $NZ = 2 \cdot 10^8$, and *irregular* mode; some experiments, such as weak scaling and the comparison between regular and irregular matrices, instead modify these parameters according to the analysis performed. The large problem size was chosen to obtain a representative computational workload and highlight more clearly the behavior of the parallel implementations during performance and scalability tests. The metrics used are:

- **Speedup:** measures how much faster a program runs when using a larger number of processors.

$$S(p) = \frac{T_{\text{seq}}}{T_{\text{par}}(p)}$$

where T_{seq} is the sequential reference time (of the provided sequential implementation), measured on the same problem and the same hardware, while $T_{\text{par}}(p)$ is the execution time of the parallel program with p processing elements.

- **Efficiency:** measures how effectively a parallel program uses the p available resources.

$$E(p) = \frac{S(p)}{p} \in [0, 1]$$

where p represents the number of processing elements.

- **Relative speedup:** measures how effectively a parallel program can use an increasing number of processors, indicating whether adding further processors improves performance.

$$S_{\text{rel}}(p) = \frac{T_{\text{par}}(1)}{T_{\text{par}}(p)}$$

- **Weak scaling efficiency:** measures how effectively the program scales when the problem size grows proportionally to the number of processing elements.

$$WSE(p) = \frac{T_{\text{par}}(PS(1), 1)}{T_{\text{par}}(PS(p), p)}$$

4.1 Shared-memory execution time and speedup

Figure 1 compares C++ Threads, with $B = 1024$, and OpenMP task-based, with $B = 512$. Each point represents the median of three runs, and speedup is calculated relative to the time with a single worker of the same implementation.

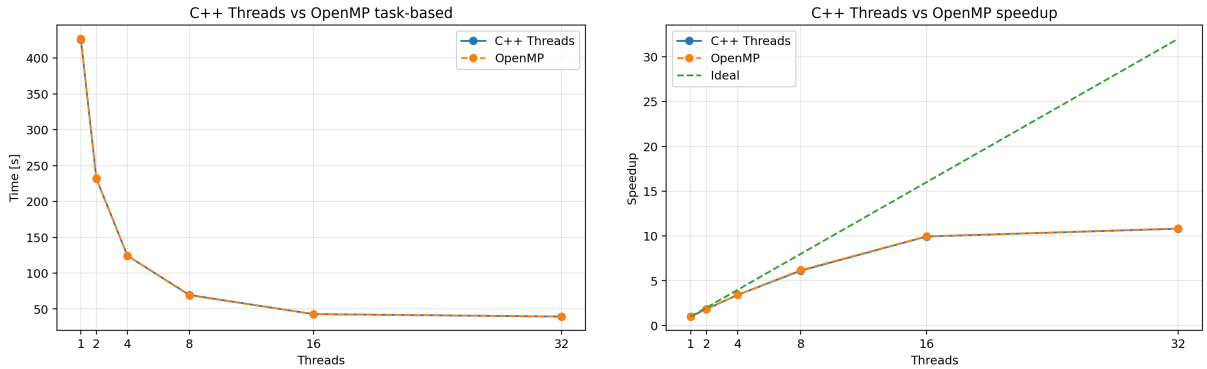


Figure 1: Comparison between C++ Threads and OpenMP task-based: median execution time (left) and speedup relative to a single worker (right).

The two implementations show nearly identical results. Moving from one to 16 workers, the time decreases from approximately 426 s to 42.8 s, with a speedup close to 9.95. With 32 workers, the time decreases only to approximately 39.4 s, and speedup reaches approximately 10.8. The limited improvement beyond 16 workers is consistent with the use of SMT and the limitations of the memory subsystem, which is shared by the logical threads

4.2 Granularity Analysis

The granularity tests highlight the trade-off in choosing the block size. Small values generate many chunks and, in the OpenMP case, many tasks, increasing scheduling overhead. Excessively

large values instead reduce this overhead, but can worsen load balance and limit the available parallelism. For this reason, an optimal intermediate value emerges in all implementations.

The granularity of the C++ Threads and OpenMP task-based versions was analyzed using block sizes ranging from 256 to 8192, with 16 and 32 threads. As shown in Table 1, in task-based SpMV the number of chunks corresponds directly to the number of OpenMP tasks generated.

Table 1: Median times as a function of granularity.

Block B	Chunk	C++ Threads		OpenMP task-based	
		16 thread	32 thread	16 thread	32 thread
256	3907	43.40	39.22	43.38	39.77
512	1954	42.91	39.20	42.92	39.35
1024	977	42.65	38.99	42.95	39.58
2048	489	42.74	39.65	43.58	40.35
4096	245	43.29	40.95	44.04	41.33
8192	123	44.45	58.67	45.21	64.42

The results lead to the selection of $B = 1024$ for C++ Threads and $B = 512$ for OpenMP task-based. With $B = 8192$, however, the problem is divided into only 123 chunks, fewer than four per thread in the 32-thread configuration. The small number of independent work units limits the ability of dynamic scheduling to compensate for workload irregularity, causing the sharp increase in execution times observed in both implementations.

The optimal granularity changes because chunks are managed differently. In C++ Threads, they are assigned dynamically to persistent workers through an atomic counter, whereas in OpenMP a task is created for each SpMV chunk.

MPI+OpenMP granularity sweep. MPI+OpenMP granularity was analyzed on 8 nodes, with one MPI rank per node, comparing 16 and 32 OpenMP threads per rank and varying the block size from 256 to 4096.

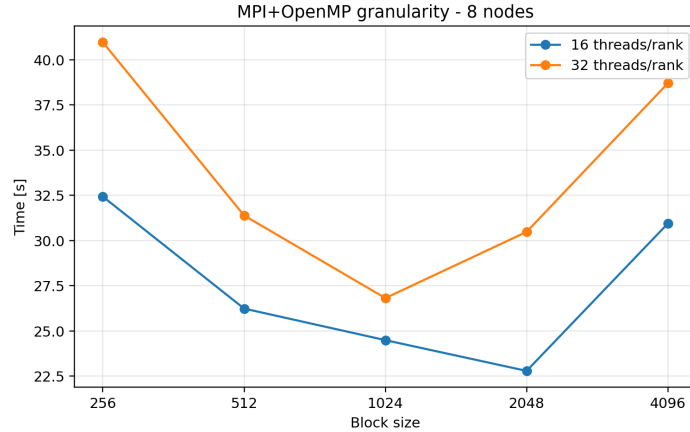


Figure 2: MPI+OpenMP granularity sweep on 8 nodes, with one rank per node and 16 or 32 threads per rank.

With 16 threads per rank, the minimum time is obtained with $B = 2048$, at approximately 22.79 s, while with 32 threads the best result is approximately 26.82 s with $B = 1024$. Small block sizes increase the number of tasks generated both in local SpMV and in the rotation of the global vector; with 16 threads, for example, rotation time decreases from approximately 14.20 s with $B = 256$ to approximately 0.51 s with $B = 2048$.

With excessively large block sizes, however, the number of local tasks decreases and the risk of load imbalance increases. With $B = 4096$, SpMV time rises to approximately 17.37 s with 16

threads and 20.32 s with 32 threads. The configuration selected for subsequent experiments is therefore $B = 2048$ with 16 threads per rank.

4.3 Comparison between regular and irregular workloads

The comparison shows that C++ Threads and OpenMP are only slightly sensitive to the transition from a regular to an irregular workload: the curves remain nearly overlapping, and no systematic penalty emerges. In both cases, chunks are dynamically scheduled across the entire matrix, allowing the runtime to compensate for differences in cost between dense and sparse regions.

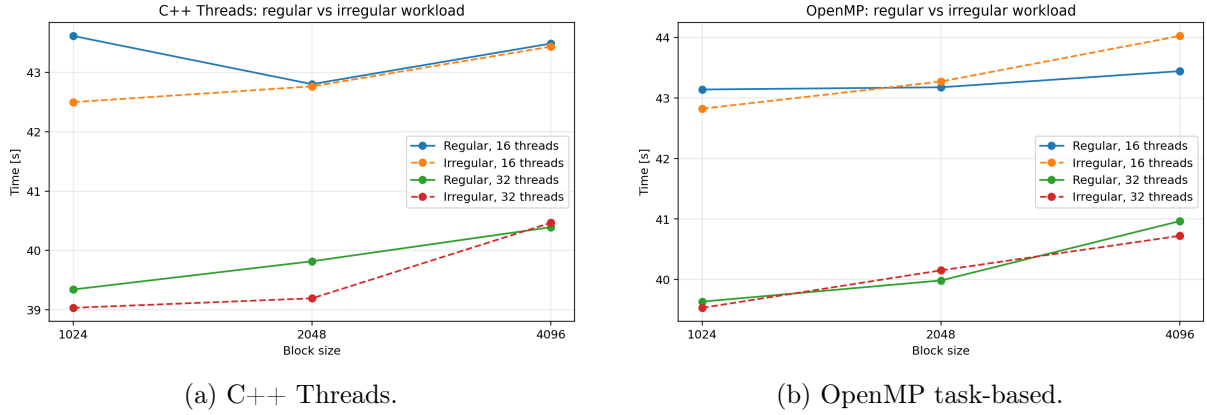


Figure 3: Comparison between regular and irregular workloads in the shared-memory implementations. The two modes show very similar times for all configurations considered.

In the MPI+OpenMP version, however, the behavior is much more sensitive to irregularity, especially as the block size increases. The MPI distribution correctly balances the ranks with respect to the number of nonzero elements, but in the irregular case this produces partitions with very different row counts. Since the subsequent OpenMP parallelism still defines tasks in terms of rows, ranks containing a small number of very dense rows may have too few tasks to use all threads effectively.

The effect becomes particularly evident with large blocks. With a `block_size` of 4096, the granularity becomes too coarse, and intra-rank parallelism decreases. In the configuration with 32 threads per rank, execution time increases from 14.99 s in the regular case to 39.06 s in the irregular case, making the latter approximately 2.6 times slower.

The penalty is further amplified by MPI synchronization: each iteration contains collective operations, so a rank that completes its local work more slowly can also delay the other processes. The result therefore indicates that the problem is not the MPI distribution of nonzero elements itself, but the interaction between a partition balanced with respect to NNZ and an OpenMP granularity still based on the number of rows.

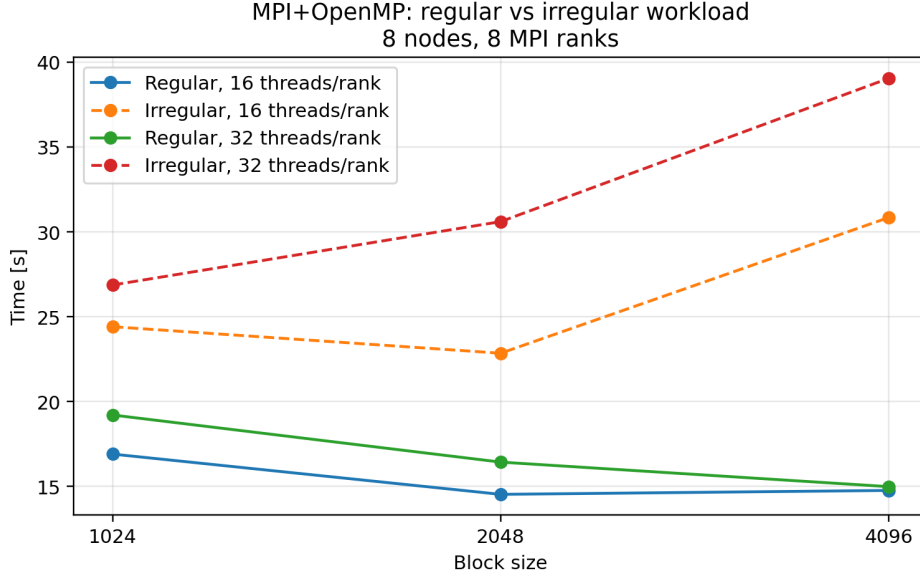


Figure 4: Comparison between regular and irregular workloads for MPI+OpenMP on 8 nodes and 8 MPI ranks. The penalty in the irregular case grows considerably as the block size increases.

4.4 Strong Scaling

Strong scaling was performed by keeping the reference problem fixed and progressively increasing the number of nodes. Following granularity tuning, a configuration with one MPI rank per node, 16 OpenMP threads per rank, and block size $B = 2048$ was used.

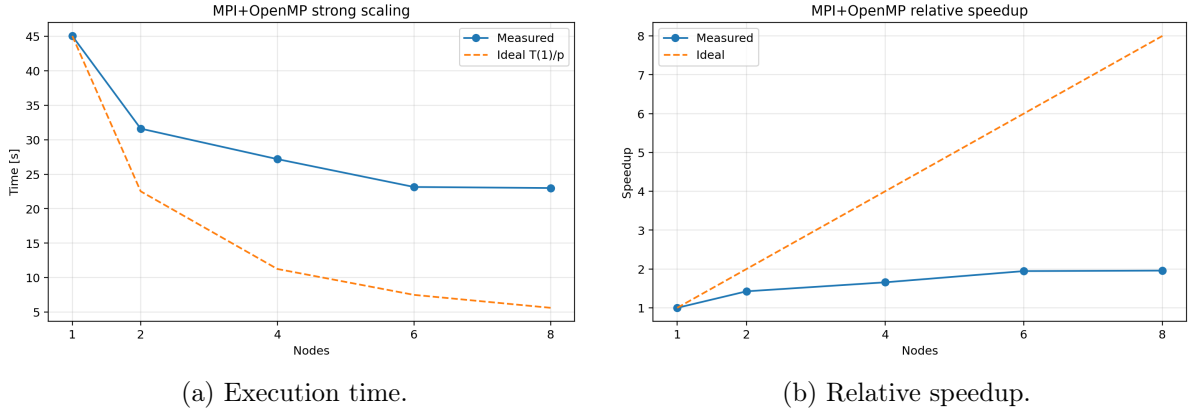


Figure 5: MPI+OpenMP strong scaling on the irregular problem.

Table 2: Strong scaling results. Absolute speedup is calculated relative to the sequential baseline of 425.759 s, while relative speedup uses the hybrid execution on one node as the reference.

Nodes	Time (s)	Abs. speedup	Rel. speedup	Rel. efficiency
1	45.04	9.45	1.00	100.0%
2	31.61	13.47	1.42	71.3%
4	27.19	15.66	1.66	41.4%
6	23.15	18.39	1.95	32.4%
8	23.00	18.51	1.96	24.5%

Execution on one node already achieves an absolute speedup of 9.45 through the use of 16

physical cores. As the number of nodes increases, the time decreases from 45.04 s to 23.00 s, but relative speedup reaches only 1.96, with an efficiency of 24.5%. The improvement is almost negligible between six and eight nodes.

This behavior results from the reduction in local computation alongside global operations whose costs do not decrease proportionally. The maximum SpMV time decreases from 41.59 s to 9.26 s because each rank processes fewer nonzeros. However, at each iteration, `MPI_Allgatherv` reconstructs the entire vector on each rank. As the number of nodes increases, each rank computes a smaller portion of the vector but must receive a larger portion from the other ranks. The time measured in `Allgatherv` therefore grows from 1.35 s to 11.26 s.

Granularity also limits the benefit of the additional nodes: the MPI partitions are balanced by nonzeros, but OpenMP tasks are blocks of 2048 rows. At eight nodes, five ranks have only 10 SpMV tasks each, which are insufficient to occupy the 16 threads simultaneously. Local computation therefore does not decrease in proportion to the resources: between six and eight nodes, SpMV time decreases only from 9.46 s to 9.26 s.

Differences in rank completion times can also translate into waiting in the subsequent `MPI_Allreduce`. Its time, approximately 7.97 s on eight nodes, includes these waits as well, rather than only the communication of the single norm scalar. The dependencies between computation and collectives, together with the limited task parallelism on some ranks, therefore explain the final plateau. Initial distribution, vector rotation, and the logical epoch update remain secondary costs.

4.5 Weak Scaling

In weak scaling, the global problem size is increased proportionally to the number of nodes, according to the relations $N = p \cdot 125\,000$ and $\text{nnz} = p \cdot 25\,000\,000$, with $p \in \{1, 2, 4, 6, 8\}$. The tests use irregular matrices, one MPI rank per node, 16 OpenMP threads per rank, and block size $B = 2048$.

The partitions are contiguous row ranges, whose boundaries are determined by the cumulative number of nonzeros. Each rank therefore receives approximately 25 000 000 nonzeros, while the number of rows may differ. The results are the medians of three runs, each with 500 iterations. The time includes the initial CSR distribution but excludes matrix generation on rank 0.

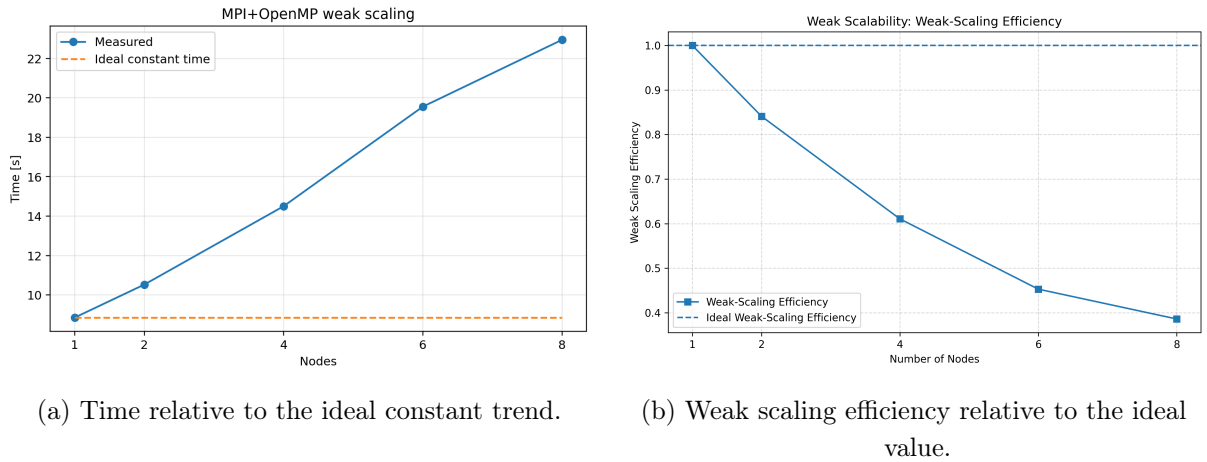


Figure 6: MPI+OpenMP weak scaling results.

Table 3: Problem sizes and MPI+OpenMP weak scaling results.

Nodes	Global N	Global nnz	Time (s)	Weak scaling efficiency
1	125 000	25 000 000	8.85	100.0%
2	250 000	50 000 000	10.53	84.1%
4	500 000	100 000 000	14.50	61.1%
6	750 000	150 000 000	19.55	45.3%
8	1 000 000	200 000 000	22.94	38.6%

The time increases because the number of nonzeros per rank remains constant, but the cost of the entire iteration does not. The eight-node point uses the same global size and the same hybrid configuration as the corresponding strong scaling test. The times, 22.94 s and 23.00 s respectively, are nearly identical and consistent across the two experiments.

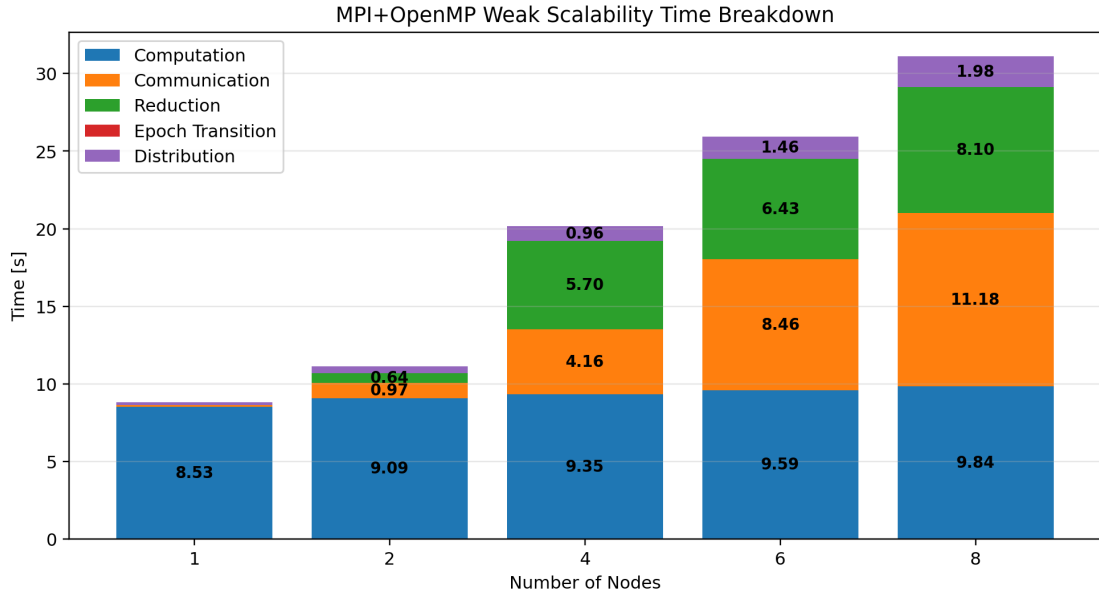


Figure 7: Phase times for MPI+OpenMP weak scaling. *Computation* includes SpMV, normalization, and rotation. The components are medians of separately computed maxima across ranks: their sum does not represent the total time.

Time breakdown. The computation component varies overall from approximately 8.53 s to 9.84 s. Specifically, the maximum local SpMV time increases from 8.32 s to 9.23 s, an increase of approximately 11%. The result confirms that nonzero-based balancing keeps the matrix workload per rank relatively stable. The residual growth is consistent with the increase in the global vector’s working set and with possible differences in the completion times of the irregular partitions.

The matrix is distributed, while the global vector is replicated: at each iteration, `MPI_Allgatherv` reconstructs on each rank a vector of size N , which grows with the number of nodes. The amount of data to receive from the other ranks therefore increases, and this cost is repeated for all 500 iterations. Since the call is blocking and the subsequent rotation requires its result, the implementation does not overlap this communication with computation. This mechanism explains the growth in the time measured in `Allgatherv`, from 0.12 s to 11.18 s.

Balancing nonzeros also does not guarantee the same utilization of the 16 physical cores available on each node, distributed across two eight-core sockets. OpenMP tasks comprise 2048 rows, regardless of their density: at eight nodes, the first five ranks receive approximately 19 500 rows each and generate only 10 SpMV tasks, which are insufficient to occupy all threads simultaneously, while the last rank generates 302 tasks. Partitions with similar numbers of nonzeros may therefore have different completion times.

These discrepancies can cause waiting in the subsequent `MPI_Allreduce`, whose time grows to 8.10 s despite combining only one `double` per rank. The increase therefore does not depend on message size: the measured time includes waiting for contributions from the other ranks and the costs of the collective, rather than only the transfer of the norm.

Each rank also retains complete copies of the vectors x , y , and the global result. Consequently, while the local SpMV workload remains almost constant, the memory occupied by the vectors and the cost of rotation grow with the global size. Rotation traverses the entire vector on each rank, rather than a partition of it: local work therefore grows with N , even though the number of threads remains fixed.

The initial CSR distribution also contributes to the growth, as it starts from rank 0 alone and must distribute an increasing global amount of data, rising from 0.17 s to 1.98 s. This cost is incurred only once. Normalization and rotation remain secondary components, while the epoch transition is negligible because it requires only the logical update of `row_shift`, without modifying or redistributing the matrix. The available data, however, do not allow the loss of efficiency to be attributed to demonstrated network or memory saturation.

For each run, the time of each phase is reduced separately through `MPI_MAX`; the median of the three runs is then calculated. Since the maxima may belong to different ranks and the phase medians may come from different repetitions, the sum of the bars does not necessarily coincide with the wall-clock time. In particular, computation on one rank can overlap with waiting on another. At eight nodes, the components sum to 31.11 s, compared with a total time of 22.94 s. The latter also includes the final diagnostic operations and the reductions of the phase counters, which are not represented separately in the breakdown. The plot makes it possible to identify the critical phases, but not to assign them exact shares of the overall time.

4.6 OpenMP task-based vs OpenMP work-sharing

The two OpenMP implementations execute the same algorithm within a persistent parallel region, but distribute work differently. In the task-based version, a `single` thread creates one task for each SpMV chunk and uses `taskwait` to separate the phases; the dot product and scaling are also divided into one task per thread. The work-sharing version instead uses `omp for schedule(dynamic, B)` for SpMV and a static distribution for vector operations.

The comparison was performed on the irregular matrix with $N = 10^6$, $\text{nnz} = 2 \times 10^8$, using 16 and 32 threads and block sizes ranging from 256 to 8192. Each value represents the median of three runs performed with the same affinity configuration.

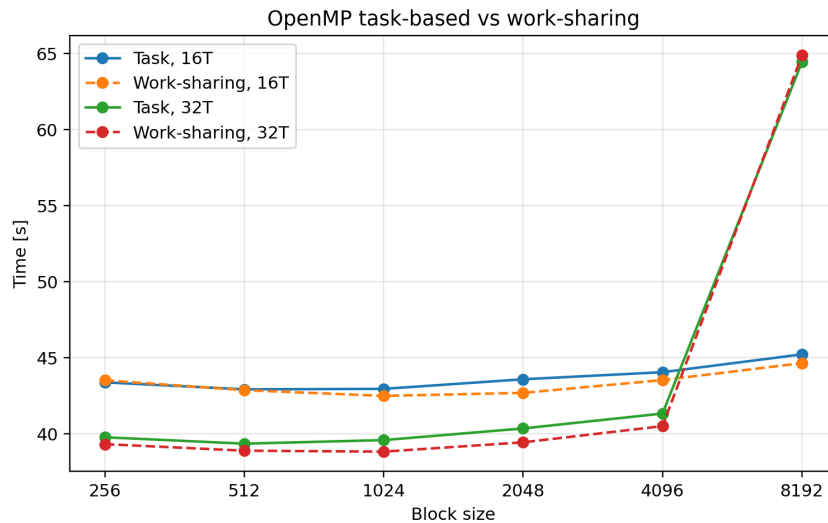


Figure 8: Comparison between OpenMP task-based and dynamic work-sharing as a function of block size.

The task-based version achieves its minimum with $B = 512$, requiring 42.92 s with 16 threads and 39.35 s with 32 threads. For work-sharing, the best value is instead $B = 1024$, with times of 42.49 s and 38.82 s, respectively. In the range $B = 512$ –4096, work-sharing is 0.1–2.3% faster, because it avoids explicit task creation and the associated work of the producer thread. The advantage nevertheless remains limited because the dominant cost is SpMV, which is characterized by irregular accesses and low arithmetic intensity.

With $B = 8192$, both versions exceed 64 s in the 32-thread configuration. In this case, only 123 chunks are available, and the reduction in parallelism, together with the imbalance of the irregular workload, has a much greater impact than the choice between tasks and work-sharing.

4.7 MPI+OpenMP configuration sweep

After selecting $B = 2048$ through the granularity analysis, a sweep of the hybrid MPI+OpenMP configuration was performed to determine how to divide resources between MPI processes and OpenMP threads.

The experiment keeps 8 nodes and a total of 128 physical cores fixed, varying the number of MPI ranks and OpenMP threads per rank. All configurations use block size $B = 2048$.

Table 4: Rank–thread sweep with a fixed budget of 128 physical cores.

MPI ranks	Thread/rank	Ranks/node	Median time (s)
128	1	16	34.50
64	2	8	33.34
32	4	4	28.03
16	8	2	27.25
8	16	1	22.85

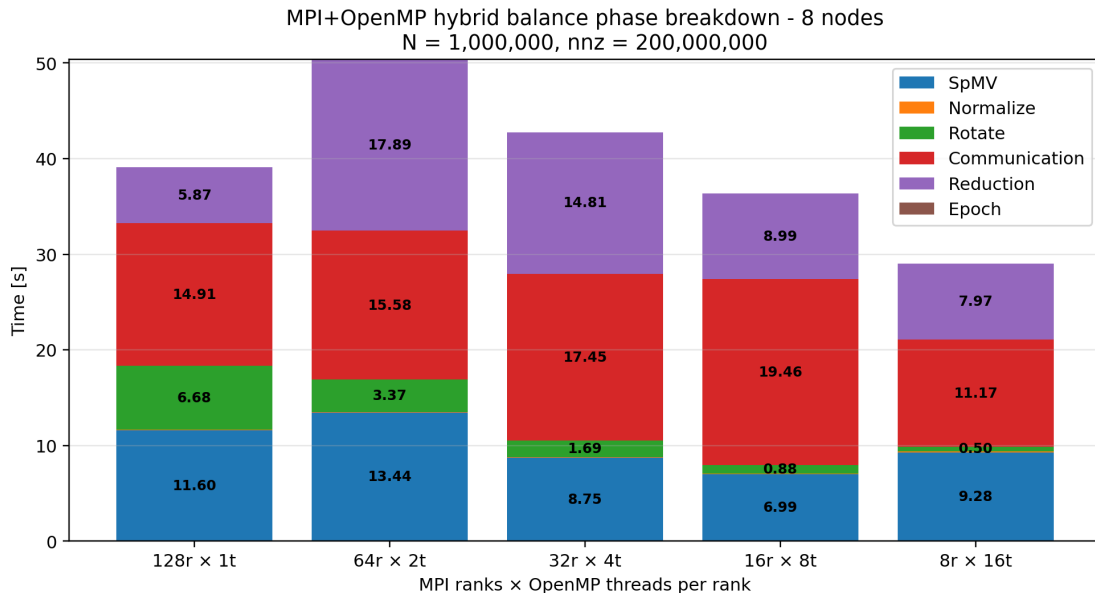


Figure 9: Phase breakdown in the rank–thread sweep. Phase times are maxima computed separately across ranks and do not constitute additive components of the wall-clock time.

The time decreases progressively when moving from configurations with many ranks and few threads to configurations with fewer ranks and more threads. It decreases from 34.50 s with 128 ranks and one thread per rank to 22.85 s with 8 ranks and 16 threads per rank.

The best configuration therefore uses a single MPI rank per node, relying on OpenMP to utilize the 16 available physical cores. Reducing the number of ranks also decreases the number of

processes involved in collective operations and local copies of the global vector, while increasing the number of threads per rank allows greater parallelization of local operations.

5 Conclusions and limitations

5.1 Conclusions

The project has shown that iterative SpMV computation on evolving sparse matrices can be parallelized correctly without physically modifying the CSR structure. Matrix evolution is represented through a logical row shift, avoiding matrix redistribution and making the cost of epoch transitions negligible.

In the shared-memory implementations, C++ Threads and OpenMP task-based exhibit nearly equivalent performance. In both cases, dynamic scheduling effectively distributes the irregular workload, provided that an intermediate granularity is chosen. OpenMP work-sharing is slightly faster than explicit tasks because it reduces the overhead associated with their creation and management.

Scalability within the node slows beyond the 16 physical cores. Using 32 logical threads yields only a limited improvement, because SpMV performs few operations per data item transferred, and performance becomes primarily constrained by memory bandwidth. The use of SMT therefore does not provide additional memory resources proportional to the number of threads.

In the MPI+OpenMP version, partitioning based on the number of nonzero elements balances the arithmetic workload assigned to ranks, but does not guarantee the same thread utilization. The sweeps identified one MPI rank per node, 16 OpenMP threads per rank, and block size $B = 2048$ as the best configuration. This configuration utilizes all available physical cores while reducing the number of participants in collective operations.

5.2 Limitations and bottlenecks

The main limitation of the distributed version is its dependence on the global vector. Each iteration requires an `MPI_Allreduce` to compute the norm and an `MPI_Allgatherv` to reconstruct the complete vector on each rank. As the number of nodes increases, local SpMV time decreases, while communication and synchronization account for an increasing share of the cost. This behavior leads to the plateau observed in strong scaling and the progressive reduction in weak scaling efficiency.

Partitioning based on nonzero elements balances the arithmetic cost, but does not guarantee uniform payloads in terms of rows. In the irregular matrix, some ranks hold a few dense rows and others many sparse rows. Since `MPI_Allgatherv` communicates the vector elements associated with local rows, the amount of data sent by the different ranks may be unbalanced, introducing additional waiting times in collective operations.

Rotation also constitutes a limitation in weak scaling. It is performed locally by each rank on the entire global vector, whose size grows with the number of nodes. Its cost remains lower than that of communication, but it does not keep local work constant as SpMV does.

A further limitation concerns granularity. Excessively small blocks generate a large number of scheduling operations, while excessively large blocks reduce the available parallelism and expose workload imbalance. This effect is particularly evident with 32 threads, when a small number of chunks is insufficient to keep all workers busy.

The experimental analysis is also limited to synthetic matrices, a single computing architecture, and a maximum of eight nodes. The evolution considered consists exclusively of a circular permutation of rows: the CSR structure and the number of nonzero elements do not change during execution. The results therefore describe the behavior of this specific evolving matrix model.

Finally, the times of individual MPI phases are obtained by considering the maximum across ranks separately for each phase. Since the maxima may come from different processes, their

sum does not necessarily coincide with the wall-clock time. The breakdown makes it possible to identify the dominant phases, but does not represent an exact additive decomposition of the overall time.